Data Structures & Algorithms — Condensed Notes

A. Arrays

Definition

A linear data structure holding elements of the same type in contiguous memory. Each element is accessed by an index, usually starting at 0.

Characteristics

- **Contiguous storage:** element i sits at a + b(i − s), where a is the start address, b the element size, s the starting index.
- **Homogeneous:** all elements share the same type and size.
- **Fixed size:** set at allocation, not easily changed.
- **Random access:** any element reached in O(1) given its index.

Advantages

- **O(1) access.**
- **Space efficient:** no pointer overhead.
- **Cache-friendly:** contiguous layout improves locality.

Limitations

- **Costly resize:** growing requires a new array and a full copy.
- **Costly insert/delete:** shifting elements (except at the end).
- **Wasted memory:** unused reserved slots stay locked.

Operations & Complexity

Operation	Description	Time
Access	Read element by index	O(1)
Update	Write element by index	O(1)
Traversal	Visit every element	O(n)
Insertion	Insert + shift right	O(n)
Deletion	Remove + shift left	O(n)
Linear Search	Scan for a value	O(n)
Binary Search	Halve range each step (sorted only)	O(log n)

Applications

- **Tabular data and records.**
- **Matrices, grids, numerical simulation.**
- **Base structure for stacks, queues, heaps, hash tables.**
- **Image storage as 2D pixel arrays.**
- **Polynomial coefficient storage.**

B. Stacks

Definition & LIFO

A stack restricts insertion and deletion to one end, the top — Last-In, First-Out (LIFO), like a pile of plates.

Operations

Usually array- or linked-list-based; top tracks the last inserted index.

```
PUSH(S, x)
1. If S.top == S.size: error "overflow"
2. Else: S.top++; S[S.top] = x

POP(S)
1. If empty: error "underflow"
2. Else: S.top--; return S[S.top+1]

PEEK(S)
1. If empty: error "underflow"
2. Else: return S[S.top]
```

Complexity

- **PUSH / POP / PEEK / IsEmpty:** O(1) each (amortized O(1) even for dynamic-array resizing).
- **Space:** O(n) — data storage plus O(1) per stacked recursive call, e.g. O(n) for recursion depth n.

C. Queues

Definition & FIFO

A queue inserts at the rear and removes from the front — First-In, First-Out (FIFO), like a waiting line.

Array-based Implementation

- **Front:** index of the element to remove next.
- **Rear:** index where the next element will be inserted.

A circular array lets front/rear wrap to reuse freed space.

```
ENQUEUE(Q, x)
1. Q[Q.tail] = x
2. if Q.tail == Q.size: Q.tail = 1
3. else: Q.tail++

DEQUEUE(Q)
1. x = Q[Q.head]
2. if Q.head == Q.size: Q.head = 1
3. else: Q.head++
4. return x
```

Complexity

- **ENQUEUE / DEQUEUE / Peek / IsEmpty:** O(1) each — pure indexing, no shifting.
- **Space:** O(n), pre-allocated in a static array.

Applications

- **Scheduling shared resources (printer, disk, CPU).**
- **Breadth-First Search traversal.**
- **Buffering streamed data in arrival order.**

D. Circular Queues

A circular queue links the array's last position back to the first, avoiding the "false overflow" of a linear queue, where freed front slots can't be reused.

Representation

- **Front:** index of the element to remove.
- **Rear:** index for the next insertion.

Wrap-around uses the modulo operator: index = (index + 1) % SIZE.

Algorithms

```
ENQUEUE:
1. If full ((FRONT==0 && REAR==MAX-1) || FRONT==REAR+1): "Queue Full"
2. If FRONT==-1: FRONT=REAR=0
3. Else: REAR=(REAR+1)%SIZE
4. Q[REAR]=DATA

DEQUEUE:
1. If FRONT==-1: "Queue Empty"
2. DATA=Q[FRONT]
3. If FRONT==REAR: FRONT=REAR=-1
4. Else: FRONT=(FRONT+1)%SIZE
5. return DATA
```

Example (size 5)

With front=2, rear=4 holding elements at 2,3,4, the queue isn't full — enqueuing wraps rear to 0, filling the vacant slot.

Advantages / Disadvantages

- **Advantages:** efficient memory reuse; predictable O(1) performance without shifting.
- **Disadvantages:** still fixed size; overflow/underflow logic is more intricate than a linear queue.

Complexity

- **Enqueue / Dequeue:** O(1).
- **Space:** O(n).

Exam points

- **Overflow:** (rear + 1) % SIZE == front.
- **Underflow:** front == −1.
- **Capacity:** a size-n circular queue often stores only n−1 elements, keeping one slot free to distinguish full from empty.

Linear Queue vs Circular Queue

Feature	Linear Queue	Circular Queue
Logic	Straight sequence	Ring — last position links back to the first
Index movement	Front & rear only increase	Indices wrap to 0 at the end
Overflow	REAR == MAX−1, regardless of front space	Only when (REAR+1)%SIZE == FRONT
Memory reuse	Dequeued space is never reused	Freed front space is reused via wrap-around

Why circular is preferred

- **No false overflow:** a linear queue reports "full" even with vacant front slots; wrap-around fixes this.

- **Predictable performance:** shifting to fix a linear queue costs O(n); the circular queue keeps O(1) inserts/deletes.
- **Resource management:** well suited to buffering and scheduling that runs continuously.

E. Overflow & Underflow in a Circular Queue (Worked Example)

Conditions

- **Overflow (full):** (FRONT==0 && REAR==MAX−1) or FRONT==REAR+1, i.e. (REAR+1)%SIZE == FRONT.
- **Underflow (empty):** FRONT==REAR==−1 (or FRONT==REAR, depending on convention).

Illustration (size 5)

- **Start:** f = r = −1 (empty).
- **Enqueue 32, 18, 58:** front → 0, rear → 2; queue = [32, 18, 58, _, _].
- **Wrap-around:** continuing to enqueue past index 4 sets rear = (4+1)%5 = 0, reusing a freed slot.
- **Dequeue:** returns 32, front → 1; if front == rear at dequeue time, both reset to −1 (queue empty).
- **Full state:** f=0, r=4 — next enqueue fails because (4+1)%5 == 0 == front.

Complexity

- **Enqueue / Dequeue:** O(1) — direct access and pointer updates.
- **Space:** O(n).

F. Linked Lists

A linked list is a sequence of nodes linked by pointers rather than physical placement; each node holds data plus a pointer (link) to the next node.

Advantages over Arrays

- **Dynamic size:** grows/shrinks at runtime.
- **Fast insert/delete:** O(1) once the position is known, versus O(n) shifting in an array.
- **No pre-allocation waste:** memory is claimed only as nodes are created.
- **Cheap reorganization:** moving pointers is cheaper than moving large data records.

Types

- **Singly Linked List (SLL):** one "next" pointer per node, ending in NULL. Forward traversal only; O(1) head insert/delete, O(n) tail delete. Lowest memory overhead; used for simple stacks/queues.
- **Doubly Linked List (DLL):** "next" and "prev" pointers. Bi-directional traversal; O(1) insert/delete at both ends. Higher memory overhead; used for browser history, LRU caches, undo/redo.
- **Circular Linked List (CLL):** last node's next points back to the first. Any node can start a traversal; no NULL checks needed, but careless loops can run forever. Used for round-robin scheduling, stream buffering, Fibonacci heaps.

Comparison

Feature	Singly	Doubly	Circular
Pointers/node	1 (next)	2 (next, prev)	1 or 2
Traversal	Forward only	Forward & backward	Forward (both if doubly circular)
End of list	NULL	NULL	Points back to head
Tail insertion	O(n)	O(1)	O(1) with a kept last-pointer
Typical use	Simple stack/queue	Browser history	Task scheduling / heaps

G. Trees & Binary Trees

Definitions

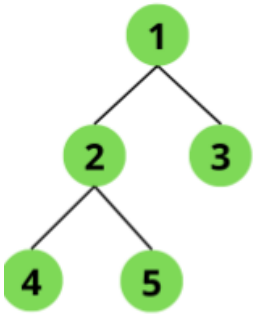
- **Tree:** a non-linear, hierarchical structure with one root node; every other node belongs to a disjoint subtree. Equivalently, a connected, acyclic undirected graph.
- **Binary Tree:** a tree where each node has at most two children, distinguished as left child and right child.

Terminology

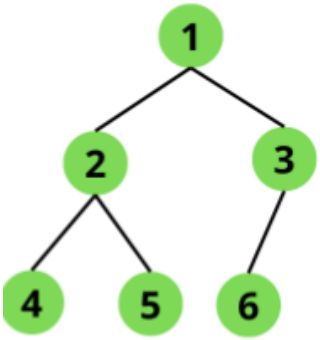
- **Root:** the single node with no parent.
- **Parent / Child:** a node with successors is a parent; each successor is its child.
- **Leaf:** a node with no children (degree 0).
- **Degree of a node:** its number of children; degree of a tree is the max over all nodes.
- **Depth:** path length from root to the node (root's depth = 0).
- **Height:** longest path from a node down to a leaf; tree height = height of the root.
- **Subtree:** a node plus all its descendants, itself a valid tree.

Types of Binary Trees

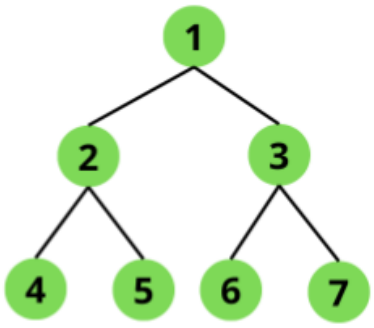
- **Full/Strict:** every node has 0 or 2 children.



- **Complete:** every level is full except possibly the last, which fills left to right.



- **Perfect:** a strict binary tree with all leaves at the same depth.



- **Binary Search Tree (BST):** for every node n, its left subtree holds values ≤ n and its right subtree holds values ≥ n, with no duplicates.
- **AVL Tree:** self-balancing BST — left/right subtree heights differ by at most one.
- **Binary Heap:** a complete binary tree satisfying the max- or min-heap property; backs priority queues.

Advantages / Disadvantages

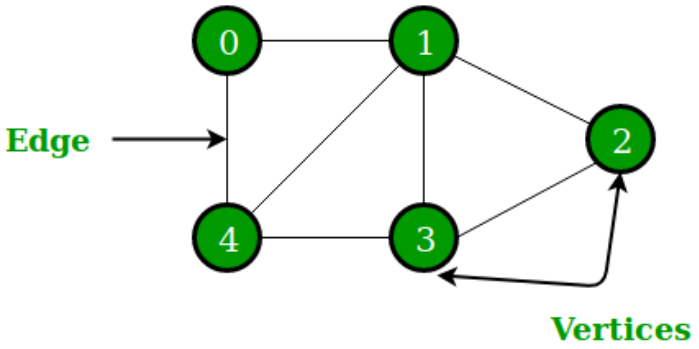
- **Advantages:** natural for hierarchies; balanced trees give O(log n) search/insert/delete; linked form allocates memory only as needed.
- **Disadvantages:** more complex to implement than linear structures; balancing (rotations) adds overhead; pointer storage and, for incomplete array-backed trees, unused slots both waste space.

Applications

- **File-system directory hierarchies.**
- **Database indexing (B-Trees, B+ Trees).**
- **Expression trees for compilers.**
- **Decision trees for classification/sorting lower bounds.**
- **Heaps for priority queues and Heapsort.**
- **Huffman coding for compression.**

H. Graphs

A graph G = (V, E) is a non-linear structure of vertices V and edges E connecting vertex pairs — more general than a tree, since any pair of nodes may connect and a node may have multiple incoming edges.



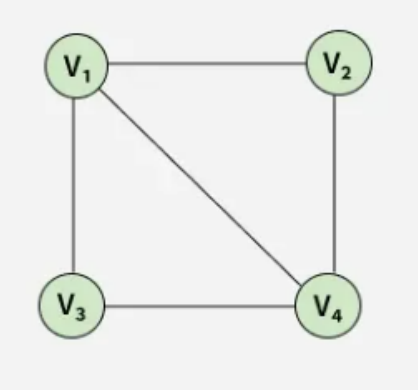
Terminology

- **Endpoints:** the two vertices an edge connects.
- **Adjacent / Incident:** two vertices linked by an edge are adjacent; the edge is incident to them.

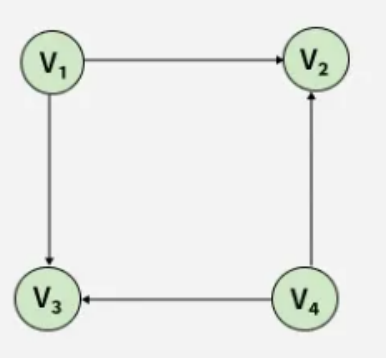
- **Degree:** edges touching a vertex (undirected); degree 0 = isolated, degree 1 = pendant.
- **Walk / Path:** a walk is any vertex-edge sequence; a path repeats no vertex or edge.
- **Cycle:** a path that returns to its starting vertex.
- **Connected:** a path exists between every vertex pair.

Types

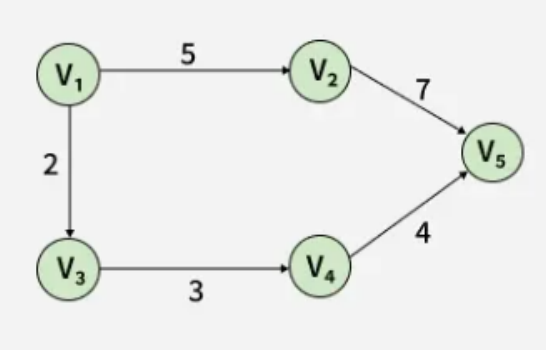
- **Undirected:** edges are unordered pairs (symmetric relation) — e.g. a two-way road network.



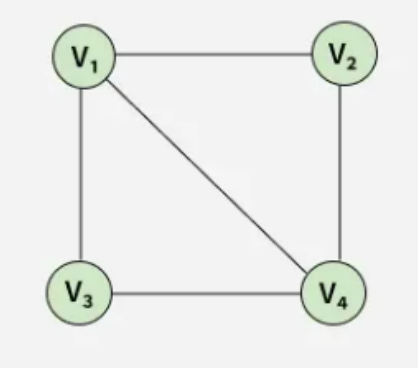
- **Directed (Digraph):** edges (arcs) are ordered pairs, with indegree/outdegree per vertex — e.g. dependency graphs in dynamic programming.



- **Weighted:** each edge carries a numeric weight (distance, cost, time) — e.g. a road map with distances.



- **Unweighted:** all edges count equally — e.g. a social network, where "shortest path" means fewest hops.



I. Adjacency Matrix vs Adjacency List

Feature	Adjacency Matrix	Adjacency List
Representation	N × N 2-D array	Array of linked lists
Space	$O(V^2)$	O(V + E)
Best for	Dense graphs	Sparse graphs (most real systems)
Edge lookup	O(1)	O(degree(v))
OS example	Theoretical protection matrix	Access-control / capability lists

Adjacency Matrix

Cell [i, j] marks whether an edge exists between nodes i and j. In an OS protection matrix, rows are domains and columns are objects, with each cell holding access rights.

- **Advantages:** O(1) lookup; conceptually simple, unified view.
- **Disadvantages:** wastes space on sparse graphs; expensive to resize.

Adjacency List

Stores only the edges that exist, typically via linked lists. OS variants: Access Control Lists (per-object list of domains) and Capability Lists (per-domain list of objects).

- **Advantages:** space proportional to actual edges; cheap to extend.
- **Disadvantages:** checking a specific relationship can require an O(n) scan of a list.

J. Divide and Conquer, and Greedy Algorithms

Divide and Conquer

A recursive design strategy with three steps at each level:

- **Divide:** split the problem into smaller subproblems of the same kind.
- **Conquer:** solve subproblems recursively, directly at the base case.
- **Combine:** merge subproblem solutions into the overall answer.

Example: Merge Sort splits, recursively sorts, then merges; Quick Sort does most of the work while partitioning and needs little combining; Binary Search is another classic case.

Greedy Algorithms

A greedy algorithm builds a solution stepwise, always taking the locally optimal choice in the hope it leads to a globally optimal result, without revisiting past decisions.

Steps

- **1. Initialize:** start with the problem's initial state and an empty solution.

- **2. Select:** evaluate the options available from the current state.
- **3. Check feasibility:** pick the best option by the objective function; keep it only if it doesn't violate constraints.
- **4. Augment:** add the chosen item to the solution and advance the state.
- **5. Terminate:** repeat until solved or no choices remain.

When it's optimal

- **Greedy-choice property:** local optimal choices lead to a global optimum.
- **Optimal substructure:** an optimal solution contains optimal solutions to its subproblems.

Greedy fails on some problems (e.g. 0-1 Knapsack, which needs dynamic programming) but succeeds for Dijkstra's shortest path, Kruskal's minimum spanning tree, and Huffman coding.

K. Backtracking and the N-Queens Problem

Definition

Backtracking builds a solution one component at a time, testing each partial solution with a bounding function. A path that can't succeed is abandoned — pruned — and the algorithm backtracks to try another alternative. It is essentially brute force refined via recursion over a state-space tree.

N-Queens via Backtracking

Place n queens on an n×n board so no two share a row, column, or diagonal.

- **Representation:** an n-tuple (x1,...,xn), where xi is the column of the queen on row i.
- **Build incrementally:** place queens row by row.
- **Check constraints:** for each row, try each column and test safety.
- **Recurse or backtrack:** advance on a safe placement; otherwise return to the previous row and try another column.

Algorithm

```
nQueens(k, n):
1. For col[k] = 1 to n:
2.   If positionOK(k):
3.     If k == n: print col[1..n]
4.     Else: nQueens(k+1, n)

positionOK(k):
1. For i = 1 to k-1:
2.   If col[k]==col[i] or |col[k]-col[i]| == k-i: return false
3. Return true
```

Worked Example (4-Queens)

- **1.** Place Queen 1 at (Row 1, Col 1).
- **2.** Queen 2: columns 1–2 are attacked; place at (Row 2, Col 3).
- **3.** Row 3: every column is attacked — backtrack.
- **4.** Move Queen 2 to (Row 2, Col 4); now Queen 3 fits at (Row 3, Col 2).
- **5.** Row 4: every column attacked — backtrack all the way to Row 1; move Queen 1 to (Row 1, Col 2).
- **6.** Continue until a valid tuple, e.g. (2, 4, 1, 3), is found.

Positions that fail the bounding function are "dead nodes" and are pruned so their children are never generated.

Complexity

- **Time:** brute force is O(n^n); backtracking's unique-column constraint reduces the search space to O(n!) permutations, though worst case remains exponential.
- **Space:** O(n) for the recursion stack and the position array.